Rule42

Working design / 1 November 2026

Shared research question

Can evolutionary search learn compact adapters that recover safe task completion around frozen world-model systems, reducing reliance on manually tuned behavioural rules?

Experiment 1: learning safe behaviour around LeWM

Page 1: method and environments

Purpose. Learn a small behavioural adapter while keeping the pretrained world model and task planner frozen. The aim is useful safe completion with less task-specific tuning. Matching an expert-designed reference could be valuable; beating it is not required. Explicit safety definitions are still needed.

Three environments. TwoRoom tests navigation and provides the first mechanism analysis. PushT tests moving and orienting the T-block. OGBench Cube tests moving the cube to its original object goal, not merely reaching with the gripper. LeWM is the opening study before the main Block Plucking study; ManiGuard is deferred.

What should carry across. Reuse the adapter family and learning recipe, training new weights for each environment. Test fresh cases within each task. Declare differences in units, action dimensions and task goals. This does not claim that the same weights transfer between different pretrained models.

Task and hazards. Keep each original task goal. Add supplied forbidden regions and no-hazard controls. Specify exactly what is protected: the navigation body, pusher, gripper or other named geometry. Cube needs an explicit vertical hazard extent. A safe gripper path alone does not prove a feasible object-moving sequence. Admit cases by a fixed procedure, not by favourable controller outcomes.

Adapter. Qualify two possible interfaces on development cases: choose an intermediate visual target, or make a bounded correction to the planner's command. The residual may be simpler to reuse across tasks and Block Plucking. Select one family before the main comparison. Do not yet fix the earlier TwoRoom-specific 112-parameter actor as the common architecture.

Learning and safety. All learners use the same actor and information within a task, with generic initialisation and no expert fallback. A passive monitor records actual violations but never vetoes actions. Remove inherited hazard masks and shields. A learned predictive safety checker is unnecessary unless it has a declared role. Training may be unsafe in simulation.

Evolutionary method. Start with CMA-ES for a compact neural actor. Evaluate task return and episode violation frequency separately. Prefer candidates meeting a stated empirical risk limit; among them, prefer higher return. Rank failing candidates by safety shortfall. This avoids a tuned safety-penalty multiplier, but small samples do not certify the true risk.

Experiment 1: comparisons, ablations and the dial

Page 2: evidence and scope

Main comparisons. Across TwoRoom, PushT and Cube, compare CMA-ES, parameter random search and PPO-Lagrangian learning the same adapter. Include the frozen planner, an untrained adapter and an expert-designed fixed reference. PPO-Lagrangian adjusts its safety multiplier during learning; it still has hyperparameters. Count every method's development and selection work.

Shared objective. Use the verified task reward and the same risk target. For RL, charge one safety cost at the first violation and zero thereafter; its undiscounted episode total records whether any violation occurred. Monitor the whole executed action prefix, including between adapter decisions. Preserve original task goals and correct termination for every method.

Focused ablations. Use TwoRoom for the fuller analysis: trained versus untrained; feasibility-based versus task-only evolution; feasibility-based versus penalised evolution; training without hazard information; and removing history or simplifying the actor if these components remain. If using target selection, examine memory and readout contributions. Retrain removal ablations. Repeat the essential learning and safety-selection checks in PushT and Cube.

Keep the workload focused. A fixed-penalty PPO sweep can test tuning sensitivity in TwoRoom. Extra optimizers, every architecture in every task, shielding and a continuous dial are not all required at once. A shielded deployment comparison is separate from the main learning result.

Safety dial. First test one fixed risk target. Then train or select a small library of controllers at different requested violation-probability limits. Initially test this in TwoRoom. Keep hazards, protected geometry and the violation definition fixed. Stricter settings request less risk; looser settings permit risk without requiring it. Lock policies and dial levels before final evaluation.

What would establish a useful dial? Measure actual risk, completion and engagement at each setting, with uncertainty. Check whether stricter requests reduce risk while retaining useful performance. Do not assume monotonicity. Identical policies, uniformly zero violations or growing inactivity alone do not establish the trade-off. Changing clearance instead is a different specification dial. One network conditioned on risk can be a later extension.

Evaluation. Use fresh source cases and repeated training runs. Report safe completion, all completion, violations, unfinished episodes and engagement with denominators. Separate training risk from final performance. Match interaction allowances and report actual simulator work, model queries and time. Keep tasks and physical hosts identifiable; repeated cases are not independent new data.

Automation and next steps. Freeze the recipe after development and count all remaining tuning. Define acceptable performance differences before claiming comparability. First qualify task correctness and adapter authority; then freeze the main comparison, focused ablations and a separately scoped dial study. Budgets, seeds, sample sizes and stopping rules remain open. This is a working design, not an execution authorisation.

Full technical detail: `FirstExperiment.md`.